

Medidas de Produtividade - Aula 6

🕒 Criado em	9 de setembro de 2026 14:56
🏷️ Tags	

1. Introdução

Em Engenharia de Programas, não basta saber se um programa funciona.

Também é necessário compreender:

- quanto esforço foi necessário para desenvolvê-lo;
- como esse esforço pode ser medido;
- se uma equipe está conseguindo evoluir o software;
- se o código está ficando mais difícil de manter;
- se as mudanças estão sendo entregues com qualidade;
- e se as métricas utilizadas realmente representam produtividade.

Essa última questão é especialmente importante.

No desenvolvimento de software, existe uma grande diferença entre:

| **aquilo que é fácil de medir**

e

| **aquilo que realmente representa produtividade.**

Por exemplo, é muito fácil contar quantos commits uma pessoa realizou.

Mas isso não significa que:

| **mais commits = mais produtividade.**

Uma pessoa pode fazer 50 commits pequenos e outra fazer 5 commits maiores e mais complexos. Apenas olhando para a quantidade de commits, não conseguimos determinar quem produziu mais valor.

Por isso, esta aula apresenta diferentes formas de medir produtividade e mostra as limitações dessas métricas.

2. O problema de medir produtividade em software

Em atividades industriais mais tradicionais, produtividade muitas vezes pode ser associada à quantidade produzida em determinado período.

Por exemplo:

┃ Uma fábrica produziu 1.000 peças em 8 horas.

É relativamente simples estabelecer uma relação entre produção e tempo.

No desenvolvimento de software, a situação é muito mais complicada.

Uma atividade de programação pode envolver:

- análise de requisitos;
- planejamento;
- pesquisa;
- projeto;
- programação;
- testes;
- revisão de código;
- documentação;
- comunicação com a equipe;
- correção de problemas;
- manutenção;
- decisões arquiteturais.

Grande parte dessas atividades não pode ser representada adequadamente por uma simples contagem de linhas ou commits.

Por isso, medir produtividade de desenvolvedores é um problema complexo.

3. Métricas clássicas de produtividade

Historicamente, várias métricas foram utilizadas para tentar medir produtividade.

Algumas delas são baseadas diretamente na quantidade de código produzido.

Apesar de serem fáceis de calcular, hoje são vistas com bastante cuidado porque podem incentivar comportamentos inadequados.

4. LOC — Lines of Code

LOC significa:

| Lines of Code

ou:

| Linhas de Código.

A ideia é utilizar a quantidade de código como uma forma de medir produção.

Por exemplo:

```
Desenvolvedor A → 500 linhas  
Desenvolvedor B → 200 linhas
```

Uma interpretação ingênua seria:

| "O desenvolvedor A foi mais produtivo."

Mas isso não necessariamente é verdade.

Imagine:

```
resultado = calcular_total()
```

Uma solução pode precisar de poucas linhas porque o desenvolvedor criou uma boa função reutilizável.

Outra solução poderia utilizar dezenas de linhas para fazer exatamente a mesma coisa.

Nesse caso:

| mais linhas não significam necessariamente mais trabalho útil.

Por isso, LOC pode ser uma métrica de tamanho ou de alteração do código, mas é uma medida muito limitada de produtividade.

5. Tipos de LOC apresentados na aula

A aula apresenta diferentes formas de contabilizar alterações no código.

Considere:

- A = linhas adicionadas;
- D = linhas removidas.

5.1 LOC adicionadas

Representa a quantidade de linhas novas introduzidas.

$LOC_{added} = ALOC_{\{added\}} = A$

Exemplo:

50 linhas adicionadas

Então:

$LOC_{added} = 50LOC_{\{added\}} = 50$

5.2 LOC removidas

Representa a quantidade de linhas excluídas.

$LOC_{deleted} = DLOC_{\{deleted\}} = D$

Exemplo:

20 linhas removidas

Então:

$LOC_{deleted} = 20LOC_{\{deleted\}} = 20$

5.3 LOC líquidas

Representa o crescimento efetivo do código.

$LOC_{net} = A - DLOC_{\{net\}} = A - D$

Exemplo:

100 linhas adicionadas
40 linhas removidas

Então:

$LOC_{net} = 100 - 40 = 60LOC_{\{net\}} = 100 - 40 = 60$

O código aumentou efetivamente em 60 linhas.

Atenção

Isso não significa que foram produzidas "60 linhas de produtividade".

Pode ter acontecido uma grande refatoração em que muitas linhas antigas foram removidas e substituídas por uma implementação muito melhor.

6. LOC tocadas

Outra medida é o total de linhas que sofreram alteração.

$LOC_{touched} = A + D$
 $LOC_{\{touched\}} = A + D$

Exemplo:

```
80 linhas adicionadas
30 linhas removidas
```

Então:

$LOC_{touched} = 80 + 30 = 110$
 $LOC_{\{touched\}} = 80 + 30 = 110$

A métrica indica o volume de alterações realizadas.

Ela pode ser útil para analisar o tamanho de mudanças, mas novamente:

| volume de alteração não é sinônimo de produtividade.

7. PLOC e LLOC

A aula também diferencia dois conceitos.

PLOC

PLOC representa:

| número de linhas de código não vazias e não comentadas.

Ou seja, procura representar melhor o código efetivamente escrito.

LLOC

LLOC representa:

| número de instruções executáveis.

Isso é diferente de simplesmente contar linhas físicas.

Por exemplo:

```
if idade >= 18:
    print("Maior de idade")
```

Dependendo da ferramenta e da linguagem, a quantidade de linhas físicas e a quantidade de instruções podem ser contabilizadas de maneiras diferentes.

O objetivo dessas métricas é tentar representar melhor o conteúdo efetivo do programa.

8. Produtividade em LOC por hora

Uma métrica simples apresentada na aula é:

$$\text{Productivity}_{\text{LOC/hour}} = \frac{\text{Ahoras trabalhadas}}{\text{trabalhadas}}$$

Por exemplo:

```
200 linhas adicionadas
10 horas trabalhadas
```

Então:

$$200/10 = 20 \frac{\text{LOC}}{\text{hora}}$$

Resultado:

| 20 LOC/hora.

Mas essa métrica possui uma enorme limitação.

Imagine dois desenvolvedores:

Desenvolvedor A

```
200 linhas
10 horas
20 LOC/h
```

Desenvolvedor B

```
100 linhas
10 horas
10 LOC/h
```

Isso não permite concluir que A foi duas vezes mais produtivo.

Talvez B tenha:

- projetado uma arquitetura melhor;
- criado uma solução mais simples;
- corrigido um problema complexo;

- escrito código mais fácil de manter;
- criado testes importantes.

Portanto:

| LOC/hora mede quantidade de código produzido, não valor produzido.

9. Métricas baseadas no Git

Com sistemas de controle de versão, como Git, tornou-se possível coletar diversas informações automaticamente.

Entre elas:

- quantidade de commits;
- quantidade de linhas modificadas;
- arquivos modificados;
- frequência das alterações;
- autores;
- histórico de mudanças.

Isso é extremamente útil para análise de projetos.

Porém, existe uma diferença importante:

| Git fornece dados sobre atividade. Isso não significa automaticamente que esses dados representem produtividade.

10. Número de commits

Uma das métricas mais fáceis de obter é:

CommitCount=nu´mero de commitsCommitCount = número\ de\ commits

Exemplo:

```
Ana → 30 commits
João → 15 commits
```

Seria errado concluir automaticamente:

| Ana foi duas vezes mais produtiva.

Um commit pode ser:

```
1 pequena correção
```

ou:

```
uma grande implementação envolvendo vários arquivos.
```

Além disso, diferentes desenvolvedores podem possuir estilos diferentes de commit.

Uma pessoa pode realizar:

```
commit → commit → commit → commit
```

enquanto outra pode trabalhar em uma tarefa e fazer:

```
commit
```

no final.

A quantidade de commits depende do fluxo de trabalho.

Conclusão

Número de commits é uma medida de atividade, não uma medida completa de produtividade.

11. Tamanho médio das alterações

Podemos calcular:

$$\text{AvgDiffSize} = \frac{A+D}{\text{CommitCount}}$$

Exemplo:

```
500 linhas adicionadas/removidas  
10 commits
```

Então:

$$\text{AvgDiffSize} = \frac{500}{10} = 50$$

Cada commit teve, em média, 50 linhas alteradas.

Essa métrica fornece mais contexto do que simplesmente contar commits.

Ainda assim, não responde:

| "Qual foi o valor produzido?"

Ela apenas descreve o tamanho médio das mudanças.

12. Arquivos modificados

Outra possibilidade é contar:

FilesTouched=nu´mero de arquivos modificadosFilesTouched = número\ de\ arquivos\ modificados

Essa informação pode ser interessante para entender a amplitude das mudanças.

Por exemplo:

```
Mudança A → 2 arquivos
Mudança B → 15 arquivos
```

A segunda alteração possui maior alcance.

Isso pode indicar:

- maior complexidade;
- maior quantidade de dependências;
- menor modularização;
- ou simplesmente uma funcionalidade que naturalmente envolve vários componentes.

Portanto, novamente:

| a métrica precisa ser interpretada dentro do contexto.

13. Métricas baseadas em testes

Uma abordagem mais interessante é observar os testes.

13.1 Testes aprovados

TestsPassed=nu´mero de testes aprovadosTestsPassed = número\ de\ testes\ aprovados

Se um projeto possui:

```
100 testes
95 passaram
```

temos:

```
95 testes aprovados
```

Isso fornece uma informação importante sobre a qualidade do software.

Mas também não é uma medida perfeita de produtividade.

14. Densidade de testes

A densidade de testes pode ser representada por:

$\text{TestDensity} = \frac{\text{número de testes}}{\text{KLOC}}$

KLOC significa:

| mil linhas de código.

Exemplo:

```
200 testes
10 KLOC
```

Então:

$\text{TestDensity} = \frac{200}{10} = 20$

Temos:

| 20 testes por KLOC.

Essa métrica pode ajudar na análise da estratégia de testes.

Porém:

| mais testes também não significa automaticamente software melhor.

Testes podem ser:

- redundantes;
- pouco relevantes;
- mal projetados;
- incapazes de verificar cenários importantes.

15. Cobertura de linhas

A cobertura procura medir quanto do código foi executado pelos testes.

Uma representação simplificada é:

$$\text{LineCoverage} = \frac{\text{linhas executadas}}{\text{linhas possíveis}}$$

Por exemplo:

```
100 linhas possíveis  
80 executadas pelos testes
```

Então:

$$\text{Coverage} = 80\%$$

Isso significa que os testes executaram 80% dessas linhas.

Mas 80% de cobertura significa que o software é bom?

Não necessariamente.

Imagine:

```
def sacar_dinheiro(valor):  
    if valor > saldo:  
        bloquear_conta()
```

Um teste pode executar a função, mas não necessariamente verificar se o comportamento produzido está correto em todas as situações.

Portanto:

Cobertura mede o quanto do código foi exercitado, não necessariamente o quanto ele foi corretamente testado.

16. A grande questão das métricas

Podemos resumir o problema assim:

Métrica	O que ela mostra	O que ela NÃO garante
Commits	Atividade no Git	Produtividade
LOC	Volume de código	Valor produzido

Métrica	O que ela mostra	O que ela NÃO garante
LOC/hora	Código por tempo	Eficiência real
Arquivos modificados	Amplitude da alteração	Complexidade exata
Testes	Atividade de testes	Qualidade total
Cobertura	Código executado	Correção completa

Esse é um dos principais ensinamentos da aula.

17. Envelhecimento ou decadência do código

Agora entramos em um conceito central.

Código também pode "envelhecer".

Mas software não envelhece da mesma maneira que um objeto físico.

Uma cadeira velha pode simplesmente sofrer desgaste físico.

Um código antigo pode continuar funcionando perfeitamente.

O problema surge quando o sistema precisa continuar evoluindo.

Ao longo do tempo, o software passa por:

- novas funcionalidades;
- correções;
- mudanças de requisitos;
- mudanças de hardware;
- mudanças de bibliotecas;
- mudanças de ambiente;
- alterações arquiteturais;
- integrações com outros sistemas.

Essas mudanças podem aumentar a dificuldade de manutenção.

18. O que é decadência de código?

A decadência de código pode ser entendida como uma perda gradual de manutenibilidade.

Com o passar do tempo, um sistema pode se tornar:

- mais difícil de modificar;

- mais caro de manter;
- mais complexo;
- mais sujeito a erros inesperados.

Isso não significa que todo código antigo necessariamente seja ruim.

O ponto é:

a evolução contínua pode acumular complexidade e tornar determinadas partes do sistema cada vez mais difíceis de alterar.

19. Um exemplo simples de envelhecimento

Imagine que temos:

```
Sistema → 10 arquivos
```

Uma nova funcionalidade exige alterações em:

```
arquivo A  
arquivo B  
arquivo C
```

Depois outra:

```
arquivo A  
arquivo B  
arquivo D  
arquivo E
```

Depois:

```
arquivo A  
arquivo C  
arquivo D  
arquivo F  
arquivo G
```

Se cada nova mudança começa a atingir cada vez mais partes do sistema, isso pode ser um sinal de que a estrutura está ficando mais difícil de manter.

20. B.1 — Histórico de mudanças frequentes

Uma das características analisadas é:

| **quantas vezes uma unidade de software foi alterada.**

Imagine:

```
Módulo A → 3 alterações  
Módulo B → 8 alterações  
Módulo C → 40 alterações
```

O módulo C merece atenção.

Por quê?

Porque alterações frequentes podem indicar:

- requisitos instáveis;
- módulo muito importante;
- concentração de responsabilidades;
- problemas de design;
- manutenção constante;
- maior possibilidade de defeitos.

Não significa automaticamente que o módulo é ruim.

É um:

| **indicador de atenção.**

21. B.2 — Amplitude das mudanças

Agora perguntamos:

| Quantos arquivos são afetados por uma alteração?

Imagine:

```
Mudança A → 2 arquivos  
Mudança B → 20 arquivos
```

A mudança B possui maior amplitude.

Quando uma alteração simples exige modificações em muitos arquivos, isso pode indicar:

- dependências excessivas;
- baixa modularização;
- responsabilidades espalhadas;
- forte acoplamento.

Um sistema bem modularizado tende a permitir que determinadas mudanças fiquem mais localizadas.

22. B.3 — Tamanho

Outro fator é o tamanho do módulo.

Quanto maior a quantidade de código de uma unidade, maior tende a ser a quantidade de elementos que precisam ser compreendidos.

Existe uma relação prática importante:

| módulos maiores frequentemente também apresentam maior complexidade.

Por isso, tamanho pode ser utilizado como um indicador para investigar possíveis problemas de manutenção.

Mas novamente:

| tamanho sozinho não determina qualidade.

Um módulo grande pode ser muito bem estruturado, enquanto vários módulos pequenos podem estar mal projetados.

23. B.4 — Idade

Aqui aparece uma ideia interessante:

| **idade do código não significa simplesmente a data de criação do projeto.**

Uma parte do sistema pode ser antiga enquanto outra foi criada ontem.

Imagine:

Arquivo A
criado há 5 anos

Mas suas linhas foram constantemente modificadas.

Enquanto:

Arquivo B
criado há 2 anos

possui grande quantidade de código que nunca foi alterada.

Por isso, a análise de idade pode considerar a idade das linhas de código.

Isso ajuda a entender:

| quais partes do sistema são antigas e quais foram recentemente modificadas.

24. B.5 — Potencial de falhas

Outra questão importante é:

| Existe alguma relação entre o histórico do código e a possibilidade de surgirem falhas?

Modelos podem utilizar informações como:

- histórico de alterações;
- idade;
- tamanho;
- quantidade de arquivos afetados;
- características das mudanças.

O objetivo é identificar regiões do sistema que podem apresentar maior risco.

A ideia não é dizer:

| "este código certamente vai falhar."

Mas:

| "com base nesses indicadores, esta região apresenta maior potencial de problemas."

25. B.6 — Esforço

Outra variável importante é o esforço necessário para realizar uma alteração.

Imagine duas funcionalidades:

Funcionalidade A

1 arquivo
50 linhas
1 módulo

Funcionalidade B

15 arquivos
500 linhas
8 módulos relacionados

É razoável imaginar que B pode exigir mais esforço.

Modelos de esforço podem considerar:

- número de arquivos;
- tamanho;
- linhas adicionadas;
- linhas removidas;
- características das alterações;
- dependências.

Isso permite tentar prever quanto esforço será necessário para determinada mudança.

26. Evidências de decadência

A pesquisa apresentada na aula procura verificar se esses indicadores realmente mostram um processo de decadência.

A conclusão apresentada é que existem evidências de que, conforme o software evolui:

- mudanças podem ficar mais espalhadas;
- alterações podem exigir mais esforço;
- a manutenção pode se tornar mais difícil;
- a manutenibilidade pode diminuir.

Portanto:

o envelhecimento do código está relacionado principalmente à evolução e à manutenção contínua do sistema.

27. Produtividade individual pode ser enganosa

A atividade proposta pelo professor pede para escolher um repositório com pelo menos 15 contribuidores e verificar os commits de cada pessoa.

Por quê?

Porque isso permite demonstrar uma coisa importante:

quantidade de commits não representa necessariamente produtividade individual.

Imagine:

Desenvolvedor	Commits
A	100
B	50
C	20

Não podemos simplesmente concluir:

A > B > C

em produtividade.

Talvez:

- A faça muitos commits pequenos;
- B faça poucas alterações grandes;
- C seja responsável por arquitetura;
- C faça code reviews;
- C resolva problemas difíceis;
- B trabalhe principalmente em testes.

A métrica de commits não captura tudo isso.

28. Por que métricas individuais podem ser perigosas?

Imagine uma empresa dizendo:

"Precisamos aumentar o número de commits por desenvolvedor."

O que pode acontecer?

As pessoas podem começar a dividir artificialmente suas alterações:

```
commit 1  
commit 2  
commit 3  
commit 4  
commit 5
```

apenas para aumentar o número.

A métrica então passa a influenciar o comportamento.

Isso é um problema clássico:

quando uma métrica vira objetivo, as pessoas podem começar a otimizar a métrica em vez do resultado real.

29. DORA

Depois das métricas tradicionais, a aula apresenta uma abordagem mais voltada para o desempenho da entrega de software:

DORA — DevOps Research and Assessment.

O foco deixa de ser:

"Quantas coisas um desenvolvedor individual fez?"

e passa a observar:

"Como a equipe consegue entregar e manter software funcionando?"

As quatro métricas apresentadas são:

1. Deployment Frequency;
2. Lead Time for Changes;
3. Change Failure Rate;
4. Time to Restore Service.

30. Deployment Frequency

É a:

frequência de deploys.

Representa quantas vezes software é disponibilizado em determinado período.

$$\text{FreqDeploys} = \frac{\text{Número de deploys}}{\text{Período}}$$

Exemplo:

```
20 deploys
4 semanas
```

Então:

$$5 = \frac{20}{4}$$

Ou seja:

| 5 deploys por semana.

A ideia é entender a capacidade de uma equipe de entregar mudanças.

Atenção

Mais deploys não devem ser interpretados isoladamente como "melhor".

Se uma equipe faz muitos deploys, mas todos causam problemas, a frequência sozinha não representa bom desempenho.

Por isso DORA utiliza várias métricas em conjunto.

31. Lead Time for Changes

É o tempo entre uma mudança ser realizada e chegar à produção.

Simplificando:

```
Commit
↓
processamento
↓
testes
↓
aprovação
↓
deploy
↓
produção
```

A fórmula apresentada é:

$$\text{LeadTime} = \text{Timestamp}_{\text{deploy}} - \text{Timestamp}_{\text{commit}}$$

Exemplo:

```
Commit → segunda-feira 10h  
Deploy → terça-feira 10h
```

Lead time:

| 24 horas.

Quanto menor o tempo, maior a capacidade de entregar rapidamente uma mudança.

32. Change Failure Rate

Agora entra a qualidade da entrega.

Pergunta:

| Quantas das mudanças implantadas causaram problemas?

A fórmula é:

$$\text{ChangeFailureRate} = \frac{\text{Mudanças com falha}}{\text{Total de mudanças}} \times 100$$

$$= \frac{\text{Mudanças com falha}}{\text{Total de mudanças}} \times 100$$

Exemplo:

```
20 deploys  
2 causaram falhas
```

Então:

$$2 \div 20 \times 100 = 10\%$$

Taxa de falha:

| 10%.

Essa métrica impede que pensemos apenas:

| "A equipe entrega muito rápido."

Também precisamos perguntar:

| "A equipe entrega com qualidade?"

33. Time to Restore Service

Também temos:

| Tempo para restaurar o serviço.

Imagine que uma alteração cause um incidente.

A equipe precisa:

```
incidente
  ↓
investigação
  ↓
correção/restauração
  ↓
serviço normal
```

O tempo entre o incidente e a restauração é medido.

Quanto menor esse tempo, maior tende a ser a capacidade de recuperação da equipe.

Essa métrica está relacionada à:

| resiliência operacional.

34. Por que DORA é melhor do que contar commits?

Compare:

Métrica individual

```
Ana → 50 commits
João → 20 commits
```

Isso nos diz pouco sobre o resultado.

Agora:

DORA

```
Deploys → frequência
Mudanças → tempo até produção
Falhas → taxa
Incidentes → tempo de recuperação
```

Essas informações estão mais próximas do desempenho do processo de entrega.

Mas ainda assim:

| DORA não deve ser interpretado como uma fórmula mágica de produtividade.

35. Um cuidado importante com DORA

Imagine duas equipes.

Equipe A

100 deploys

Equipe B

20 deploys

Não podemos concluir automaticamente:

| A é melhor.

É necessário analisar:

- tamanho das mudanças;
- contexto;
- arquitetura;
- qualidade;
- tipo de sistema;
- frequência necessária;
- incidentes;
- tempo de recuperação.

Métricas precisam ser interpretadas dentro do contexto.

36. SPACE

O framework SPACE surge justamente para combater a ideia de que produtividade pode ser resumida em um único número.

SPACE representa:

- **S — Satisfaction and Well-being**
- **P — Performance**
- **A — Activity**
- **C — Communication and Collaboration**
- **E — Efficiency and Flow**

Em português:

| Satisfação, Desempenho, Atividade, Comunicação/Colaboração e Eficiência/Fluxo.

A grande ideia é:

| **produtividade de software é multidimensional.**

37. S — Satisfaction

A primeira dimensão é:

| **Satisfação e bem-estar.**

Pergunta:

| Como os desenvolvedores estão vivenciando o trabalho?

Podemos considerar:

- satisfação;
- percepção do ambiente;
- realização;
- bem-estar;
- sinais de esgotamento;
- intenção de permanecer na equipe.

Por que isso importa?

Porque uma equipe extremamente pressionada pode produzir muito no curto prazo, mas apresentar problemas posteriormente.

38. P — Performance

Performance significa observar:

| **quais resultados o trabalho está produzindo.**

Podemos observar:

- qualidade;
- confiabilidade;
- manutenibilidade;
- saúde do serviço;
- estabilidade;
- resultados para o negócio.

Perceba a diferença:

Activity → o que foi feito
Performance → qual resultado foi obtido

39. A — Activity

Activity significa:

| **atividade realizada.**

Aqui entram métricas como:

- commits;
- pull requests;
- deploys;
- alterações.

Essas métricas não são inúteis.

O problema é tratá-las como se fossem a produtividade completa.

Elas respondem:

| "Quanto aconteceu?"

Mas não necessariamente:

| "Foi importante?"

40. C — Communication and Collaboration

Desenvolvimento de software é trabalho coletivo.

Por isso, comunicação e colaboração são importantes.

Podemos observar:

- qualidade do feedback;
- code reviews;
- colaboração;
- troca de conhecimento;
- comunicação entre equipes;
- onboarding.

Imagine um desenvolvedor que resolve problemas sozinho, mas não documenta nada e não compartilha conhecimento.

Pode haver impacto negativo para toda a equipe.

41. E — Efficiency and Flow

Aqui observamos:

a capacidade de avançar no trabalho sem bloqueios e interrupções desnecessárias.

Podemos analisar:

- tempo de espera;
- handoffs;
- interrupções;
- bloqueios;
- tempo entre etapas;
- fluxo de trabalho.

Exemplo:

```
Desenvolvedor termina tarefa
↓
espera 3 dias por revisão
↓
espera 2 dias por aprovação
↓
espera 1 dia pelo deploy
```

O desenvolvedor pode ter trabalhado bem.

O problema está no fluxo.

Portanto:

| nem sempre uma baixa produtividade aparente é causada pelo desenvolvedor.

Pode ser causada pelo próprio processo da organização.

42. DevEx — Developer Experience

DevEx significa:

| **Developer Experience — Experiência do Desenvolvedor.**

O foco é entender:

| **como é, na prática, trabalhar dentro daquele ambiente de desenvolvimento.**

Isso inclui as ferramentas, processos, arquitetura e condições de trabalho.

O conceito é muito importante porque uma equipe pode possuir excelentes desenvolvedores e ainda assim produzir lentamente devido às dificuldades do ambiente.

43. As três dimensões principais do DevEx

O material apresenta três dimensões:

1. Ciclos de feedback

Quanto tempo demora para o desenvolvedor descobrir se aquilo que fez funcionou?

Exemplos:

- testes;
- builds;
- CI;
- code review;
- aprovação;
- deploy.

Se um teste demora 30 minutos para executar, o desenvolvedor recebe feedback lentamente.

2. Carga cognitiva

É a quantidade de esforço mental necessária para entender e trabalhar no sistema.

Imagine um projeto com:

- arquitetura confusa;
- documentação ruim;
- muitas tecnologias;
- responsabilidades pouco claras;
- código difícil de entender.

O desenvolvedor precisa gastar energia tentando descobrir:

└ "Como esse sistema funciona?"

Antes mesmo de implementar a funcionalidade.

Isso aumenta a carga cognitiva.

3. Estado de fluxo

É a capacidade de trabalhar de maneira contínua e concentrada.

Interrupções constantes prejudicam esse fluxo.

Exemplo:

```
programando
↓
mensagem
↓
reunião
↓
problema urgente
↓
mensagem
↓
reunião
↓
volta para o código
```

Mesmo que a pessoa tenha trabalhado várias horas, pouco tempo pode ter sido realmente produtivo em termos de concentração.

44. O grande valor do DevEx

DevEx tenta responder:

| "Por que estamos tendo dificuldade para produzir?"

Por exemplo:

Problema

Build demora muito.

Causa

Ciclo de feedback lento.

Ação

Melhorar CI/build.

Problema

Desenvolvedores não entendem o sistema.

Causa

Carga cognitiva elevada.

Ação

Melhorar:

- arquitetura;
 - documentação;
 - ferramentas;
 - responsabilidades.
-

Problema

Equipe é constantemente interrompida.

Causa

Fluxo de trabalho prejudicado.

Ação

Reduzir interrupções e proteger períodos de concentração.

45. Developer Thriving

Outro modelo apresentado é:

Developer Thriving

A ideia é observar condições que ajudam os desenvolvedores e equipes a trabalhar bem e se desenvolver.

O framework apresentado trabalha com quatro fatores:

1. Agency;
 2. Motivation and Self-efficacy;
 3. Learning Culture;
 4. Support and Belonging.
-

46. Agency — Agência

Agência representa a possibilidade de o desenvolvedor:

- expressar opiniões;
- discordar;
- participar de decisões;
- discutir métricas;
- conversar abertamente com a liderança.

Uma equipe onde ninguém pode questionar decisões tende a ter menos capacidade de identificar problemas.

47. Motivation and Self-efficacy

Envolve:

| motivação e confiança na própria capacidade.

Uma pessoa pode possuir conhecimento técnico, mas estar completamente desmotivada.

Da mesma maneira, alguém pode estar motivado, mas não possuir confiança para resolver problemas complexos.

Por isso, os dois aspectos são considerados.

48. Learning Culture

Uma equipe saudável precisa aprender.

Isso inclui:

- estudar;
- experimentar;
- compartilhar conhecimento;
- realizar treinamentos;
- aprender novas tecnologias;
- discutir descobertas.

Software muda constantemente.

Consequentemente:

| a capacidade de aprender também é importante para a produtividade de longo prazo.

49. Support and Belonging

É o sentimento de:

| receber apoio e fazer parte da equipe.

Isso envolve:

- colaboração;
- respeito;
- apoio;
- integração;
- aceitação;
- ambiente saudável.

A ideia central é que produtividade não depende apenas de ferramentas e código.

Ela também depende do ambiente em que as pessoas trabalham.

50. Métricas saudáveis × métricas de vaidade

Essa é uma das partes mais importantes da aula.

Métricas de vaidade

São métricas que parecem impressionantes, mas possuem pouco poder para explicar o resultado real.

Exemplos:

- quantidade de commits;
- horas trabalhadas;
- linhas de código criadas.

Por serem fáceis de contar, elas podem parecer úteis.

51. Por que uma métrica de vaidade é perigosa?

Imagine uma empresa que estabelece:

| "Cada desenvolvedor precisa produzir 500 linhas por semana."

O que pode acontecer?

O desenvolvedor pode:

- escrever código desnecessário;
- evitar refatorações;
- não remover código antigo;
- dividir mudanças artificialmente.

Resultado:

| a métrica aumentou, mas o software pode ter piorado.

Isso mostra a diferença entre:

| **otimizar uma métrica**

e

| **melhorar o sistema.**

52. Métricas saudáveis

O material apresenta como exemplos:

Lead Time for Changes

Quanto tempo uma mudança leva até ser entregue.

Cycle Time

Quanto tempo uma tarefa permanece em execução.

Código manutenível criado

Não interessa apenas produzir código.

É importante produzir código que possa ser compreendido e modificado posteriormente.

53. Cycle Time

Cycle time é:

o intervalo entre o momento em que uma tarefa entra em desenvolvimento e o momento em que está pronta para entrega.

Exemplo:

```
Tarefa começa:  
segunda-feira 9h  
  
Tarefa termina:  
quarta-feira 15h
```

O cycle time representa o tempo decorrido entre esses dois momentos.

É uma medida do fluxo da tarefa.

54. Lead Time × Cycle Time

É importante não confundir.

Cycle Time

Foca no período em que a tarefa está sendo trabalhada.

```
Início do desenvolvimento  
↓  
tarefa  
↓  
Pronta
```

Lead Time

Pode considerar o tempo desde a solicitação/commit até a entrega, dependendo da definição adotada.

Na aula, o lead time foi apresentado especificamente como:

```
commit → produção
```

A ideia geral é medir quanto tempo uma mudança leva para chegar ao usuário.

55. Por que não existe uma métrica perfeita?

Porque desenvolvimento de software é multidimensional.

Uma equipe pode:

- entregar rapidamente;
- possuir código ruim;
- ter muitos incidentes;
- trabalhar sob pressão;
- ter baixa satisfação.

Outra pode:

- entregar mais lentamente;
- produzir código sustentável;
- possuir poucos incidentes;
- manter boa colaboração;
- ter um ambiente saudável.

Qual é melhor?

Não podemos responder olhando apenas para um número.

Precisamos analisar o contexto.

56. A ideia central da aula

Podemos organizar tudo em uma evolução:

```
LOC  
↓  
Commits  
↓  
Testes
```

↓
Métricas de Git
↓
DORA
↓
SPACE
↓
DevEx
↓
Visão multidimensional

A evolução representa uma mudança de pensamento.

Inicialmente:

| "Quanto código foi produzido?"

Depois:

| "Quantas atividades foram realizadas?"

Depois:

| "Como o software está sendo entregue?"

E finalmente:

| "Como podemos compreender produtividade considerando resultados, qualidade, fluxo, colaboração e experiência dos desenvolvedores?"

57. Relação entre as aulas

Essa aula não está isolada.

Ela se conecta diretamente às anteriores.

Complexidade

A complexidade ciclomática ajuda a entender:

| quão complexo é determinado código.

Envelhecimento

A análise de envelhecimento ajuda a entender:

| como o código se comporta ao longo da evolução.

Produtividade

As métricas de produtividade tentam entender:

| como o trabalho de desenvolvimento está acontecendo.

DORA

Analisa:

| capacidade de entrega e operação.

SPACE

Analisa:

| produtividade de maneira multidimensional.

DevEx

Analisa:

| condições e experiências que afetam a capacidade de desenvolver.

Portanto, todas essas ideias fazem parte de uma mesma preocupação:

| **como desenvolver e manter software de forma eficiente, sustentável e com qualidade.**

58. O que o professor quer que você questione

Uma interpretação muito importante da aula é:

| **não aceite uma métrica simplesmente porque ela produz um número.**

Sempre pergunte:

1. O que essa métrica realmente mede?
2. O que ela não mede?
3. Ela pode ser manipulada?
4. Ela representa atividade ou resultado?

5. Ela deve ser analisada individualmente ou em equipe?
6. Qual é o contexto?
7. Qual comportamento essa métrica pode incentivar?

Essa forma de pensar é muito importante no mercado de trabalho.

59. Exemplo prático completo

Imagine duas equipes.

Equipe A

```
500 commits  
10.000 LOC  
100 PRs
```

Equipe B

```
200 commits  
3.000 LOC  
40 PRs
```

Se olharmos apenas para atividade:

| A parece mais produtiva.

Agora adicionamos outras informações:

Métrica	Equipe A	Equipe B
Commits	500	200
LOC	10.000	3.000
Lead Time	5 dias	1 dia
Falhas em mudanças	15%	3%
Restauração	8 horas	1 hora
Código manutenível	baixo	alto

Agora a interpretação muda completamente.

A equipe B produz menos código, mas:

- entrega mais rapidamente;
- possui menos falhas;

- recupera incidentes mais rapidamente;
- mantém melhor o código.

Portanto:

| **menos atividade aparente pode representar melhor resultado.**

60. O que você precisa saber para uma prova

Se a questão pedir:

"Por que número de commits não representa produtividade?"

Resposta:

Porque commits representam atividade registrada no sistema de versionamento, mas não representam necessariamente a complexidade, o valor, a qualidade ou o impacto do trabalho realizado.

"Por que linhas de código não são uma boa medida de produtividade?"

Porque produzir mais linhas não significa necessariamente produzir mais valor. Uma solução melhor pode utilizar menos código, e código excessivo pode até aumentar a complexidade e o custo de manutenção.

"O que é envelhecimento de código?"

É a degradação gradual da manutenibilidade do software durante sua evolução, fazendo com que alterações possam exigir mais esforço, atingir mais partes do sistema e aumentar o risco de problemas.

"O que é DORA?"

É uma abordagem para analisar o desempenho da entrega e operação de software, utilizando métricas como frequência de deploy, lead time de mudanças, taxa de falha e tempo de restauração.

"O que é SPACE?"

É um framework que entende produtividade como algo multidimensional, considerando satisfação, desempenho, atividade, comunicação/colaboração e eficiência/fluxo.

"O que é DevEx?"

É uma abordagem centrada na experiência dos desenvolvedores, buscando identificar fatores que facilitam ou dificultam o desenvolvimento, especialmente ciclos de feedback, carga cognitiva e estado de fluxo.

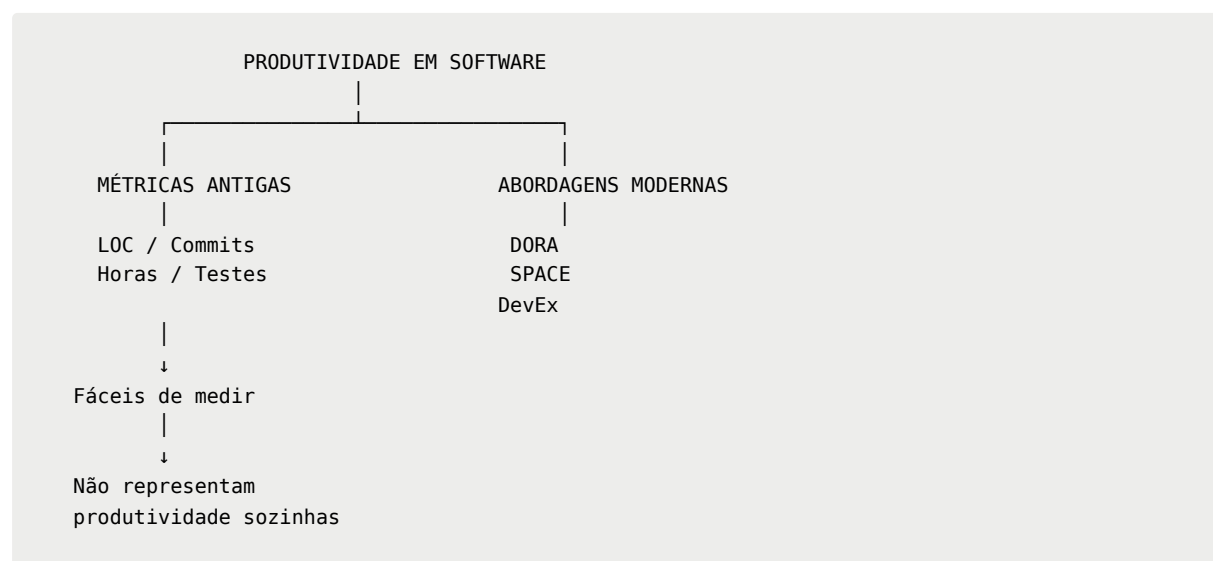
"O que são métricas de vaidade?"

São métricas fáceis de acompanhar e que podem parecer representar produtividade, mas que podem fornecer uma visão superficial do trabalho.

Exemplos:

- commits;
- horas trabalhadas;
- linhas de código.

61. Resumo visual para memorizar



E:

DORA

- Deployment Frequency
- Lead Time for Changes
- Change Failure Rate
- Time to Restore Service

SPACE

- Satisfaction
- Performance
- Activity

- └─ Communication / Collaboration
- └─ Efficiency / Flow

DevEx

- └─ Feedback loops
- └─ Cognitive load
- └─ Flow state

62. Frase principal para guardar

Se fosse necessário resumir toda a aula em uma única ideia:

Produtividade em desenvolvimento de software não deve ser reduzida à quantidade de código ou de atividades realizadas; ela precisa ser analisada considerando resultados, qualidade, fluxo, colaboração, manutenibilidade e as condições em que o trabalho é realizado.

E existe uma segunda frase igualmente importante:

Uma métrica não é necessariamente ruim porque é simples; o problema acontece quando ela é utilizada como representação completa de algo que é muito mais complexo.

63. Ligação com o mercado de trabalho

Esse conteúdo é particularmente importante porque empresas reais precisam tomar decisões sobre equipes.

Imagine uma liderança perguntando:

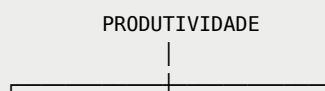
"Por que essa equipe está demorando tanto?"

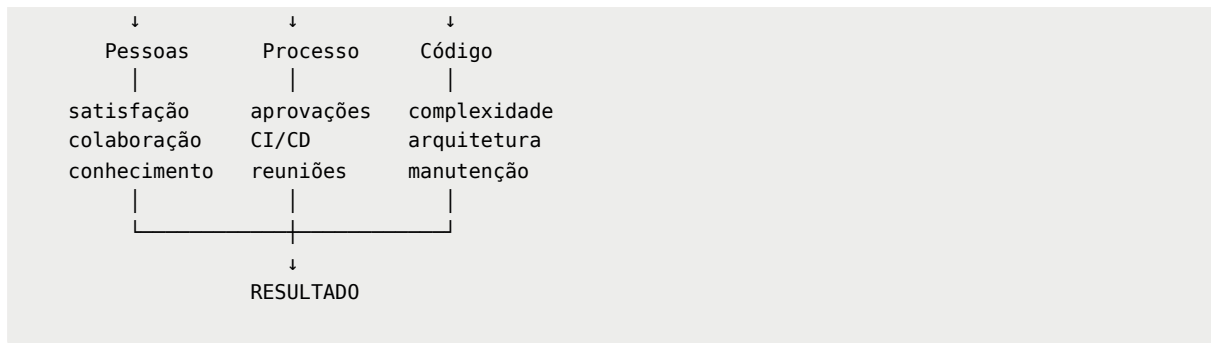
Uma análise superficial poderia olhar:

número de commits

e culpar os desenvolvedores.

Uma análise mais profissional investigaria:





Talvez o problema não seja:

| "os desenvolvedores estão trabalhando pouco."

Talvez seja:

| "o pipeline demora duas horas."

Ou:

| "as revisões levam três dias."

Ou:

| "a arquitetura exige alterar quinze arquivos para implementar uma mudança simples."

Ou:

| "a equipe está constantemente sendo interrompida."

É justamente esse tipo de pensamento que métricas mais modernas tentam estimular.

64. Conclusão

A principal evolução apresentada nesta aula é a mudança de uma visão simplista de produtividade para uma visão sistêmica.

No início, podemos tentar medir:

```
linhas de código  
commits  
horas
```

Essas métricas são fáceis de obter, mas possuem limitações.

Depois, podemos observar:

```
testes
cobertura
histórico de mudanças
idade do código
amplitude das alterações
```

Essas informações ajudam a compreender melhor o estado do software.

Por fim, frameworks como:

```
DORA
SPACE
DevEx
Developer Thriving
```

ampliam a análise para:

- resultados;
- qualidade;
- entrega;
- colaboração;
- satisfação;
- fluxo;
- ambiente de trabalho;
- capacidade de aprendizado;
- manutenibilidade.

A conclusão mais importante é que **não existe uma métrica única capaz de explicar produtividade de software.**

Uma boa análise combina diferentes evidências e, principalmente, considera o contexto de cada equipe e projeto.

Como observação final do docente:

Não existem soluções prontas que funcionem igualmente para todas as equipes.

A métrica ou abordagem adequada depende do contexto, dos objetivos, do tipo de sistema, da equipe e dos problemas que se deseja compreender.